



Configuring Ansible on AWS

Automating Cloud Infrastructure with Ansible

Introduction

AWS provides scalable cloud computing services, but **managing cloud resources manually can be complex and time-consuming**. Ansible simplifies AWS management by **automating resource provisioning, configuration, and monitoring**.

In this guide, you will learn how to:

- **Install and configure Ansible for AWS**
- **Set up AWS credentials in Ansible**
- **Use Ansible Playbooks to manage AWS resources**
- **Troubleshoot common issues**

Why Configure Ansible on AWS?

Feature	Benefit
Agentless	No need to install additional software on AWS instances.
Infrastructure as Code (IaC)	Automate and manage AWS resources with code.
Security	Uses IAM roles and AWS API securely.
Scalability	Works across multiple AWS regions and accounts.
Multi-Cloud Support	Supports AWS, Azure, GCP, and on-premises environments.



💡 *Ansible provides a simple and efficient way to automate AWS cloud management!*

Step 1: Install Ansible on Your System

Before configuring Ansible for AWS, install the required packages on your control node.

For Ubuntu/Debian

```
sudo apt update  
sudo apt install -y ansible  
pip install boto boto3 botocore
```

For CentOS/RHEL

```
sudo yum install -y epel-release  
sudo yum install -y ansible  
pip install boto boto3 botocore
```

For macOS

```
brew install ansible  
pip install boto boto3 botocore
```

Verify the installation:

```
ansible --version
```

💡 *Ensure you have Python installed as Ansible depends on it!*



Step 2: Configure AWS Credentials for Ansible

Ansible interacts with AWS using the **Boto3** Python library, which requires AWS credentials.

1. Create an AWS IAM User (If Not Already Done)

- Go to **AWS Management Console** → **IAM** → **Users** → **Create User**
- Assign the user **Programmatic Access**
- Attach the **AdministratorAccess** policy
- Note the **AWS Access Key ID** and **Secret Access Key**

2. Store AWS Credentials on the Control Node

Create a credentials file:

```
mkdir -p ~/.aws  
nano ~/.aws/credentials
```

Add the following:

```
[default]  
aws_access_key_id = YOUR_ACCESS_KEY  
aws_secret_access_key = YOUR_SECRET_KEY  
region = us-east-1
```

Verify AWS connectivity:

```
aws s3 ls
```



Using IAM roles instead of access keys is a more secure approach!

Step 3: Configure Ansible to Use AWS

Ansible needs **ansible.cfg** to define AWS-specific configurations.



1. Modify **ansible.cfg**

Edit the Ansible configuration file (**ansible.cfg**):

```
[defaults]
inventory = ./aws_ec2.yml
remote_user = ubuntu
host_key_checking = False
log_path = ./ansible.log

[privilege_escalation]
become = True
become_method = sudo
become_ask_pass = False
```

💡 *This configuration ensures Ansible can interact with AWS resources efficiently!*

Step 4: Using Ansible Playbooks to Manage AWS Resources

Once Ansible is configured for AWS, you can create Playbooks to automate cloud management.

Example: Launching an AWS EC2 Instance (launch-ec2.yml**)**

```
---
- name: Launch an AWS EC2 Instance
  hosts: localhost
  gather_facts: no
  tasks:
```



- name: Create an EC2 instance

amazon.aws.ec2_instance:

name: "MyAnsibleEC2"

key_name: "my-key"

instance_type: "t2.micro"

image_id: "ami-12345678"

region: "us-east-1"

count: 1

network:

assign_public_ip: true

register: ec2_info

- name: Display EC2 Instance Details

debug:

msg: "Instance ID: {{ ec2_info.instances[0].instance_id }}, Public IP: {{ ec2_info.instances[0].public_ip_address }}"

Run the Playbook:

ansible-playbook launch-ec2.yml



This Playbook creates an EC2 instance and displays its public IP!

Step 5: Creating a Dynamic AWS Inventory

Instead of manually listing AWS instances in an Inventory file, use **Dynamic Inventory** to fetch instances automatically.



1. Install the AWS Inventory Plugin

```
pip install ansible[amazon]
```

2. Create an AWS Inventory File (**aws_ec2.yml**)

```
plugin: amazon.aws.aws_ec2
regions:
  - us-east-1
filters:
  instance-state-name: running
keyed_groups:
  - key: tags.Name
    prefix: aws_
```

3. List Available Hosts from AWS

```
ansible-inventory -i aws_ec2.yml --list
```



*Dynamic Inventory helps manage **hundreds of AWS instances***

dynamically!

Best Practices for Configuring Ansible on AWS

1. **Use IAM Roles Instead of Static Keys** – Secure authentication using IAM roles.
2. **Store Variables Securely** – Use **Ansible Vault** for sensitive data.
3. **Use Dynamic Inventory for AWS** – Fetch AWS instance lists dynamically.
4. **Enable Logging and Debugging** – Use **-vvv** for troubleshooting.



5. Automate with CI/CD Pipelines – Integrate with Jenkins, GitHub Actions, or GitLab CI/CD.

💡 Following best practices ensures **secure and scalable AWS automation!**

Common Issues and Troubleshooting

Problem	Solution
<code>botocore.exceptions.NoCredentialsError</code>	Ensure AWS credentials are configured correctly.
<code>UnauthorizedOperation</code>	Check IAM role permissions for EC2, S3, or IAM.
<code>Instance not found</code>	Ensure the correct region and instance ID are used.
<code>Connection timeout</code>	Verify security group and firewall settings .

Know More (FAQs)

1. Can I configure multiple AWS regions?

Yes! Modify `aws_ec2.yml` to include multiple regions:

```
regions:
```

- us-east-1
- us-west-2

2. How do I fetch details of all running EC2 instances?

Run:



```
ansible-inventory --list -i aws_ec2.yml
```

3. Can I automate AWS database configuration?

Yes! Use **Ansible RDS modules** to create AWS RDS databases.

4. How do I delete an AWS resource using Ansible?

Change **state: absent**. Example:

tasks:

```
- name: Delete an S3 bucket
  amazon.aws.s3_bucket:
    name: my-ansible-bucket
    state: absent
```

5. Can I integrate Ansible with Terraform for AWS?

Yes! Use **Terraform for provisioning** and **Ansible for configuration management**.

Conclusion

Configuring Ansible on AWS **simplifies cloud automation**. With **proper setup, dynamic inventory, and Playbooks**, IT teams can **provision, configure, and manage AWS resources effortlessly**.

👉 Try configuring **AWS services** using Ansible today!